Designing AI-Driven Enterprise Systems

Agentic Patterns with Spring AI

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

Arnaud Jean
AWS



Abstract

Designing AI-Driven Enterprise Systems: Agentic Patterns with Spring AI

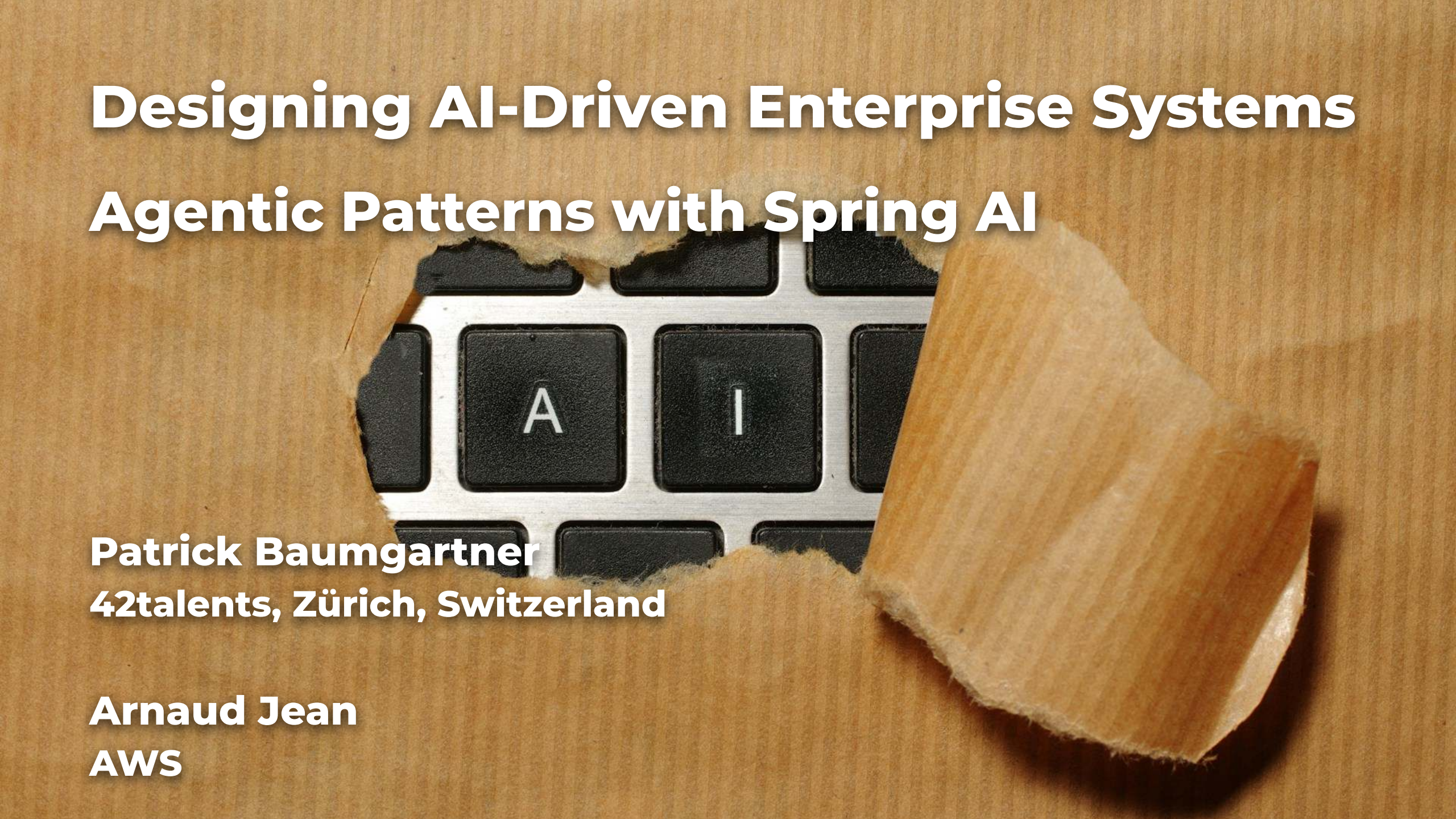
How do you introduce AI into enterprise systems without compromising architectural integrity? This session explores how Spring AI enables structured, production-grade AI integration using emerging agentic design patterns. We examine how AI becomes a first-class architectural component, not an isolated capability, within enterprise systems.

We'll explore key multi-agent patterns—such as orchestrators, swarms and graph-based execution flows—and when they make sense. We show how model-driven steps compose with deterministic business logic in Spring Boot. MCP, A2A and RAG provide context for connecting agents, tools and knowledge sources.
We will also touch on running local models to meet security and compliance requirements. Finally, we address one of the most challenging aspects of AI systems in production: testing and validating non-deterministic behaviour, and what that means for established engineering practices.

This isn't a feature catalogue; it's a practical guide to software architecture patterns and trade-offs you can apply when designing real-world Spring-based AI systems.

Designing AI-Driven Enterprise Systems

Agentic Patterns with Spring AI



Patrick Baumgartner
42talents, Zürich, Switzerland

Arnaud Jean
AWS



Designing AI-Driven Enterprise Systems Agentic Patterns with Spring AI

Patrick Baumgartner / 42talents

Arnaud Jean / AWS

Who Are We?



Patrick Baumgartner

- Technical Agile Coach @ **42talents**
- Focus on the **development of software solutions with humans**
- Coaching, Architecture, Development, Reviews, Training
- Lecturer @ **Zurich University of Applied Sciences ZHAW**
- Co-organiser of **Voxxed Days Zurich** and **JUG Switzerland**, Java Champion, Oracle ACE Pro Java, Microsoft MVP, ...
- Connect with Patrick on LinkedIn, GitHub, and @patbaumgartner on other socials

Arnaud Jean

- Senior Developer Advocate @ **AWS**
- Helps developers build cool stuff, learn faster, and have fun along the way
- Frequent conference speaker and hands-on builder
- Brings infectious energy to live coding sessions and workshops
- Connect with Arnaud on LinkedIn, GitHub, and @thewitcherish on other socials

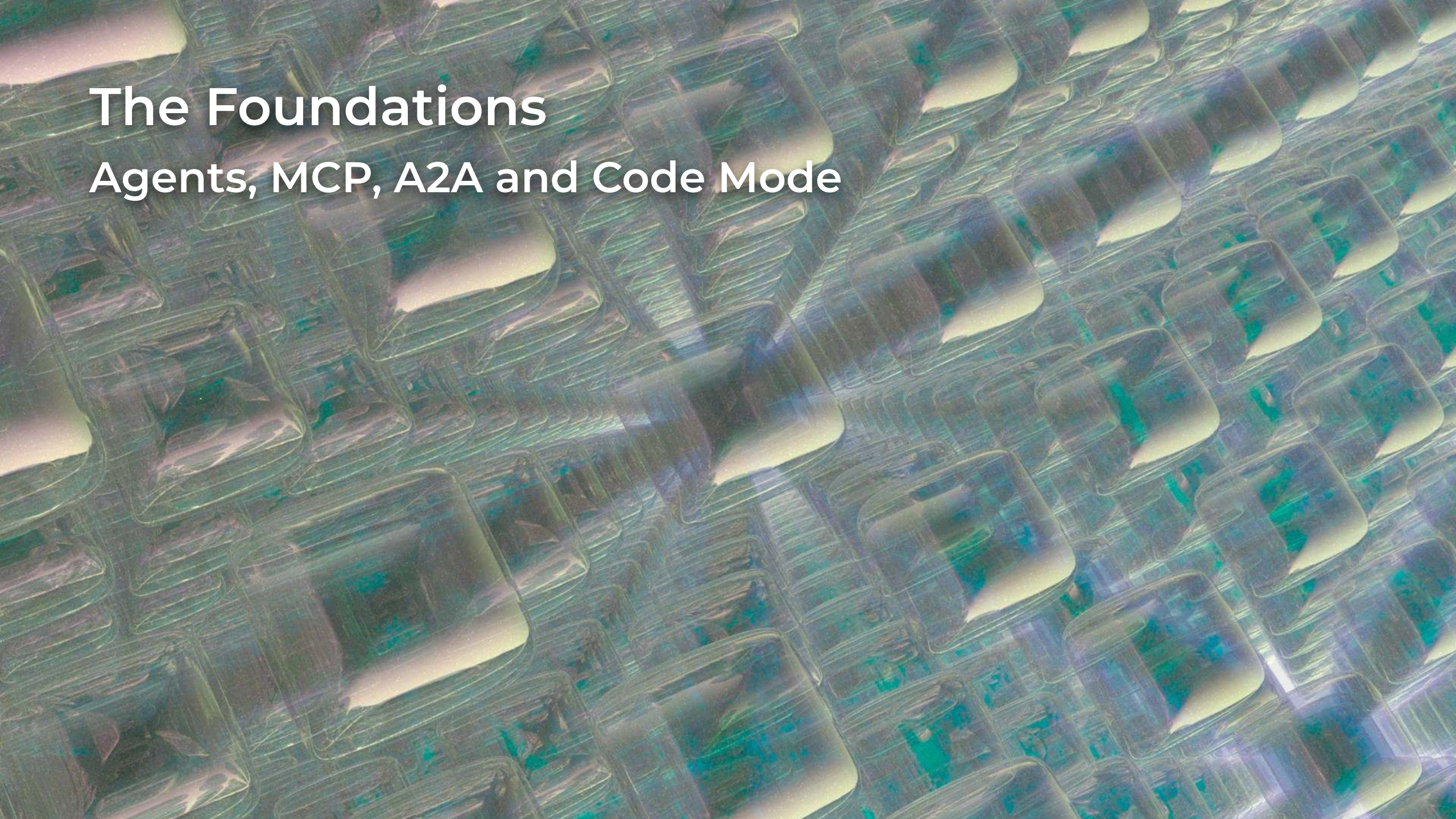
Agenda


FIELD
48 Page Memo Book
Durable Materials / Made in the U.S.A.
SALISBURY DISTRICT
WILTON
SALISBURY
FJALLRAVEN
Ordnance Survey
Outdoor Leisure
New Forest
Showing Southampton, Ringwood
Ferndown, Lymington
Christchurch

Agenda

1. **The Foundations** | Agents, MCP, A2A and Code Mode
2. **The Toolbox** | Spring AI, cloud or local models, Prompt Templates and Structured Output
3. **The Atom** | Task Execution: the LLM reads, code decides
4. **The Patterns** | Mono, Graph, Orchestrator and Swarm: from control to autonomy
5. **The Tests** | Non-deterministic agents, deterministic tests
6. **The Decision** | Pick the pattern, don't default to it

> One question guides us: **who decides what runs next?**


The Foundations
Agents, MCP, A2A and Code Mode

What Is an Agentic Workflow?

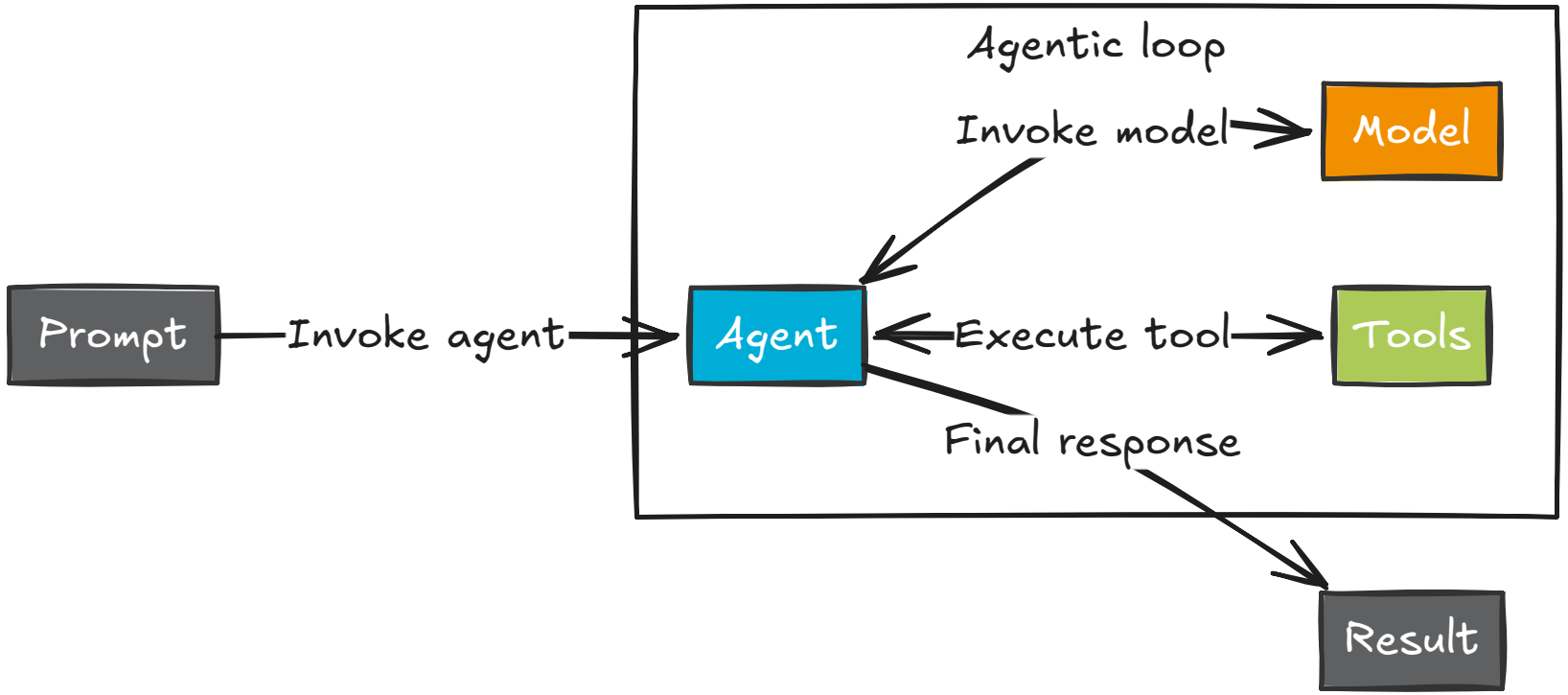
Workflows vs. Agents

	Workflow	Agent
Who decides the next step	Your code	The model, in a loop
Predictability	High: testable, auditable	Lower: emergent behaviour
Best for	Well-defined, repeatable tasks	Open-ended problems, unknown steps

> **Workflow**: your code picks the path. **Agent**: the model does.

What Is an Agent?

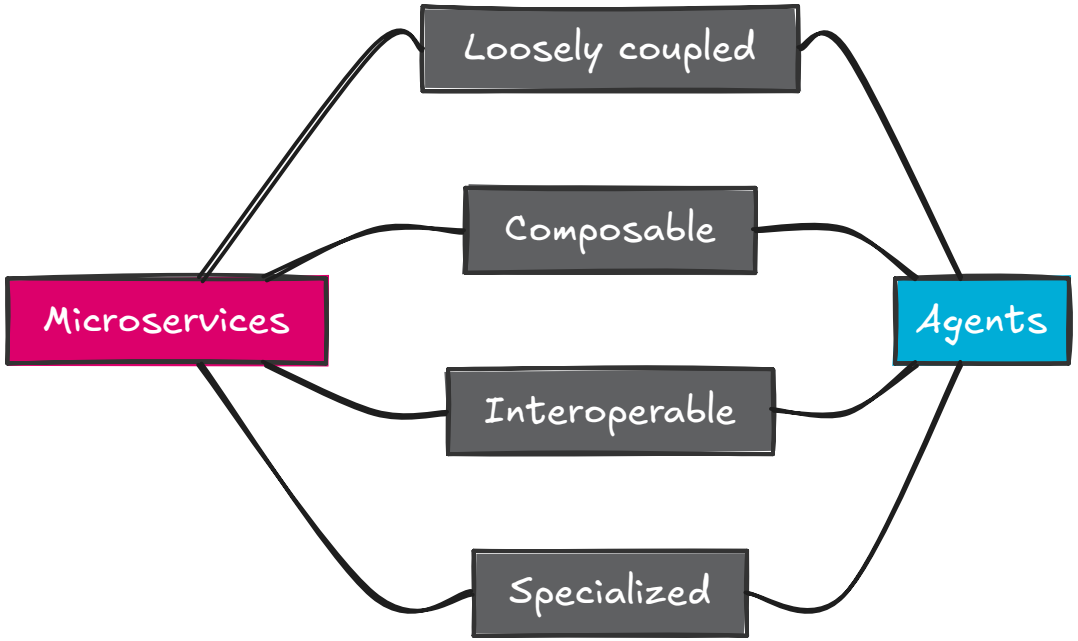
The Agentic Loop



An agent calls the **model** and its **tools** in a loop until it has a **final response**.

Microservices and Agents

Same Principles, New Building Blocks

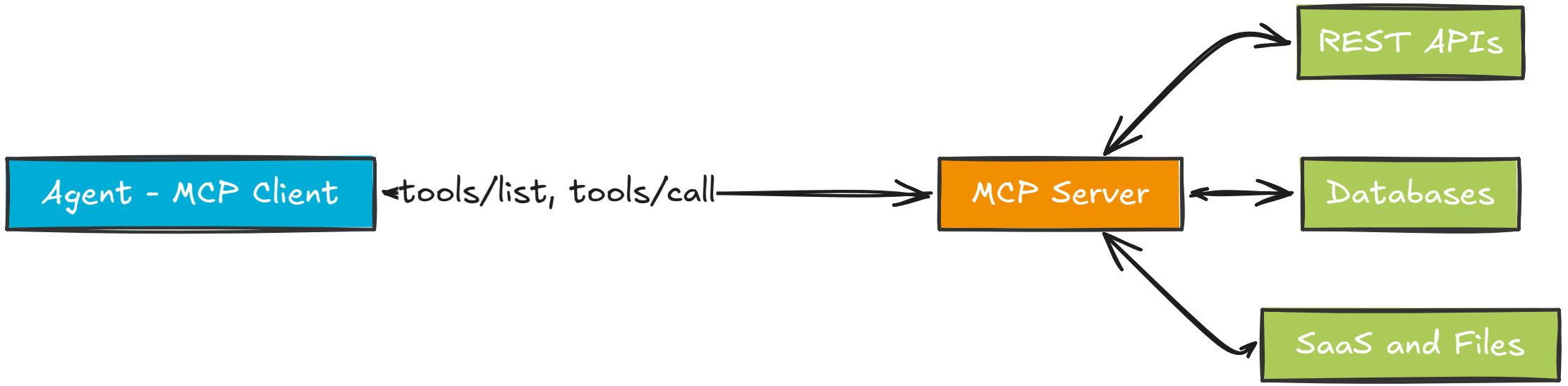


What made microservices work makes agents work: **loosely coupled**, **composable**, **interoperable**, **specialised**.

What Is MCP?

Model Context Protocol

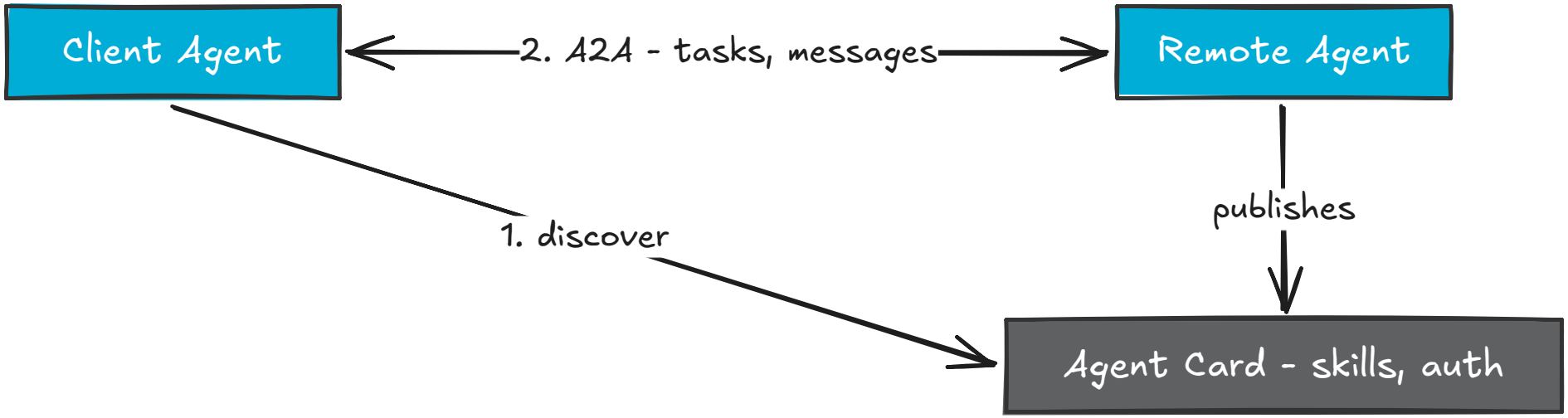
- One standard way for agents to reach **APIs, data and third-party tools**
- **Like HTTP, but for AI agents**



What Is A2A?

Agent2Agent Protocol

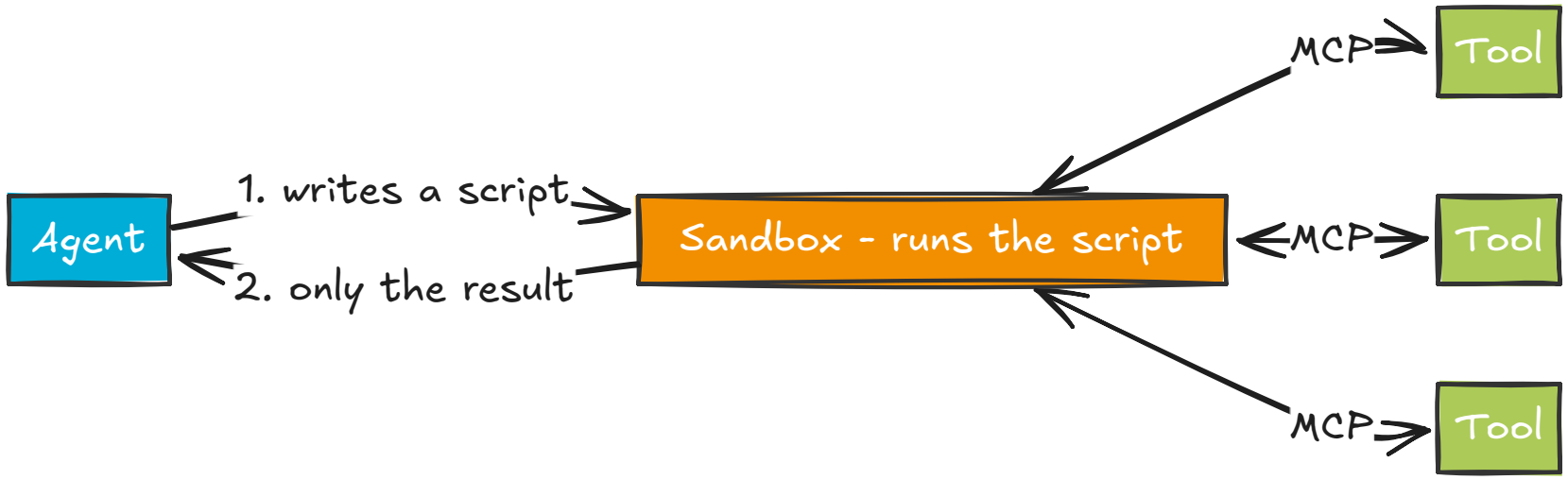
- One standard way for agents to **discover each other** and **collaborate**
- Agents stay specialised, yet work together across **teams, frameworks and vendors**



What Is Code Mode?

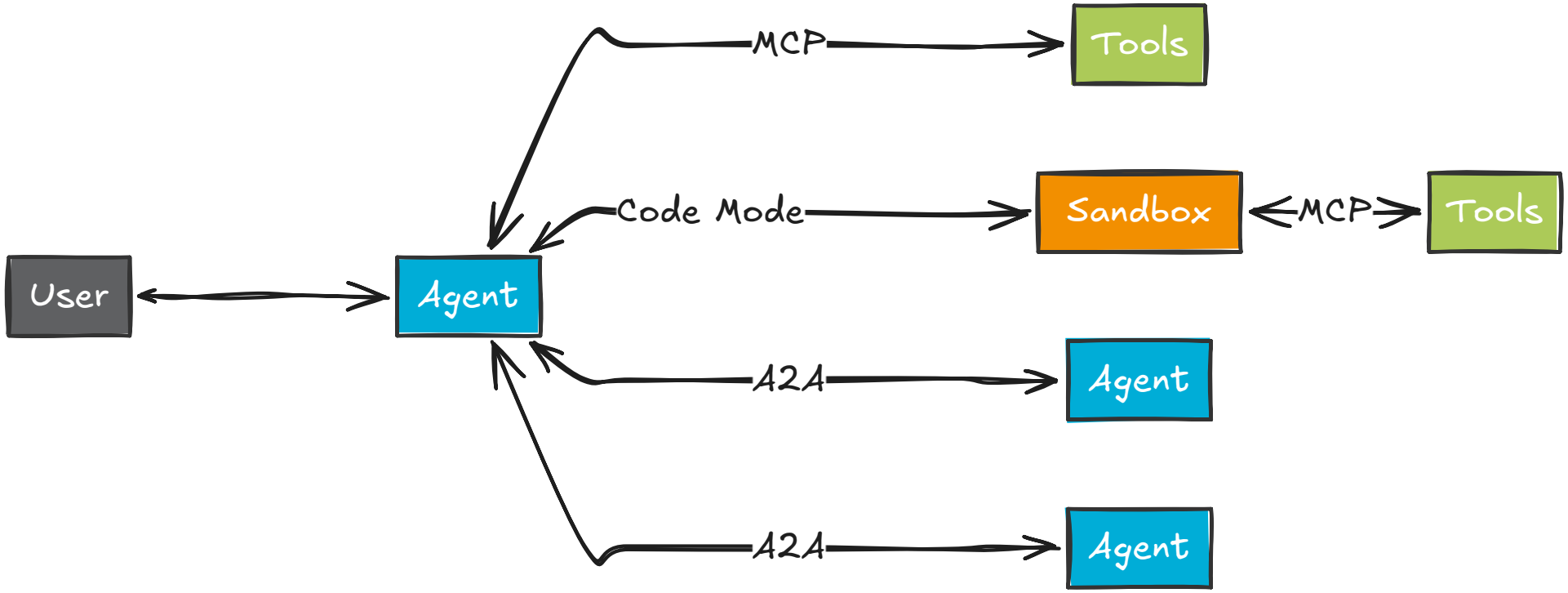
Let the Model Script Your Tools

- The model writes **one script** against a **typed tool API**, not one tool call per step
- A **sandbox** runs it; only the **final result** returns to the context window



What Is an Agent?

Tools via MCP or Code Mode, Peers via A2A



- A retrieval tool can fetch relevant passages to ground an answer (**RAG**)

The Toolbox

Spring AI the Swiss Army Knife



Spring AI + Community

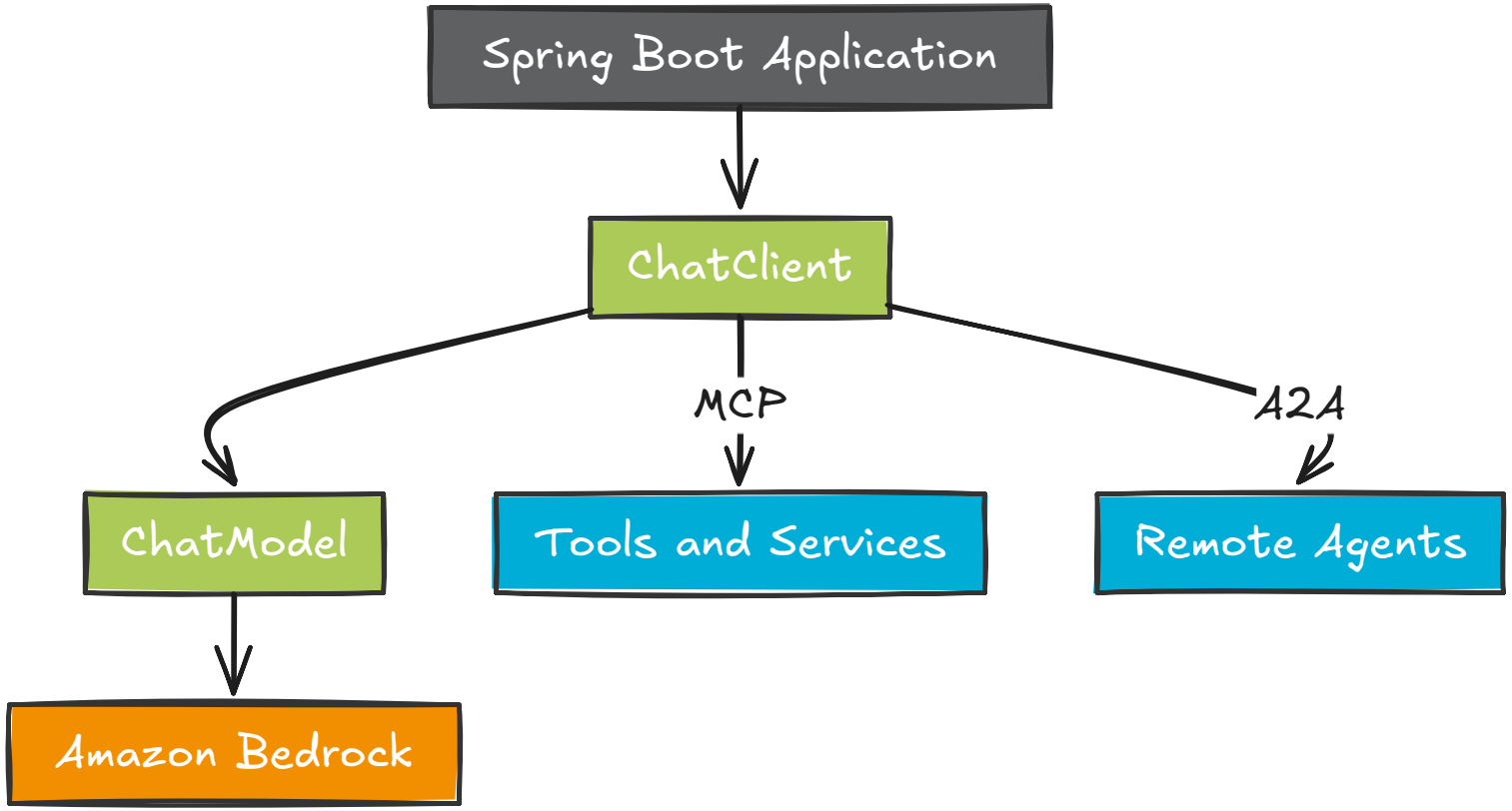
Spring for the AI Domain

- Spring's design principles, applied to **AI applications**
- Portable `ChatClient` and `ChatModel` APIs, tool calling, advisors
- Built-in **MCP** and **A2A** support
- Spring AI Community projects, such as **Spring AI AgentCore SDK**



Spring AI Building Blocks

One API, Many Models, Tools and Agents



One Codebase, Cloud or Local Models

Switch the Provider With a Spring Profile, Not a Rewrite

```
# application-bedrock.properties
spring.ai.model.chat=bedrock-converse
spring.ai.bedrock.aws.region=eu-central-1
agentic.models.fast=eu.anthropic.claude-haiku-5-5
agentic.models.smart=eu.anthropic.claude-opus-5-5
```

```
# application-ollama.properties
spring.ai.model.chat=ollama
spring.ai.ollama.base-url=http://localhost:11434
agentic.models.fast=qwen3:8b
agentic.models.smart=qwen3:14b
```

- **Local inference** can keep prompts on your own hardware, provided the model, tools, logging, and storage all remain local.
- **Amazon Bedrock** supports EU data-residency requirements by routing **EU geographic inference profiles** only across EU AWS Regions.
- The trade-off: small models are weaker at **tool calling** and **structured output**, so test each pattern on the model you ship

Right Model per Job

Match Capability, Latency and Cost to the Task

Task	Prioritise	Evaluate
Extraction and classification	Low latency, schema adherence	Accuracy and missing fields
Planning and synthesis	Reasoning and context handling	Task completion and tool use
Evaluation	Clear criteria and critical review	Agreement with human assessments

- Choose the **least costly model that meets your quality target**
- Configure model choices separately from workflow logic
- Evaluate the judge separately; a different model can provide another perspective

> Validate model choices against **representative inputs**.

Technique | Prompt Template

Prompts Are Code: Treat Them Like Code

```
Goal: Extract the document title and key
points into the requested JSON structure.

Requirements:
- Copy the title exactly as printed.
- Summarise the key points concisely.
- Use only facts from the document.
- Return an empty title if none is present.

Document:
<<<
{document}
>>>
```

```
String response = chatClient.prompt()
    .user(u -> u
        .text(new ClassPathResource(
            "prompts/extract-document.st"))
        .param("document", document))
    .call()
    .content();
```

- Spring AI renders `{placeholders}` with **StringTemplate**
- Structure: **Goal**, **Requirements**, **Steps**
- Be explicit: list every rule
- `.st` files are versioned and **reviewed like code**

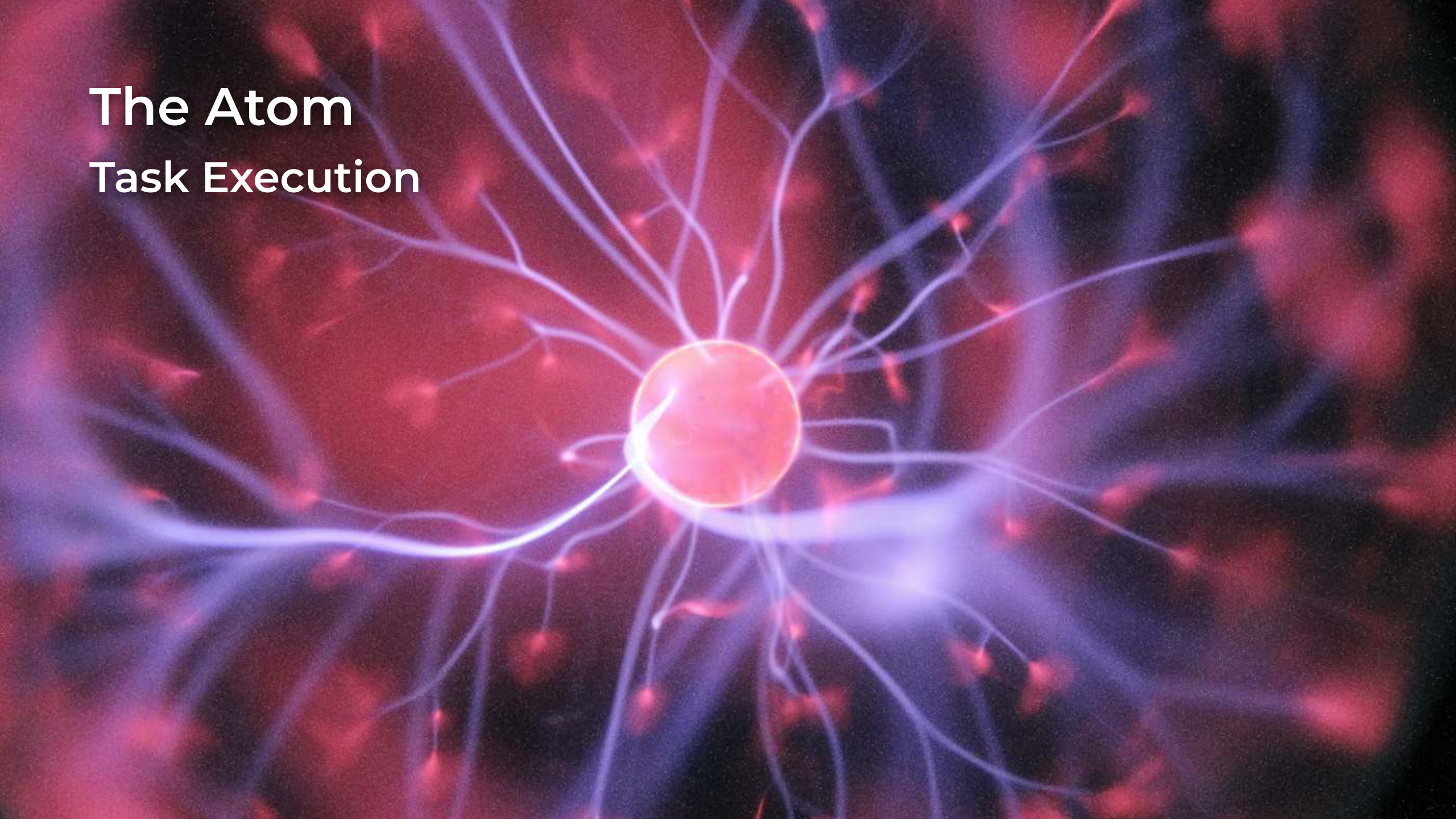
Technique | Structured Output

Stop Parsing Strings

```
record ExtractedContent(
    @JsonPropertyDescription("Document title exactly as printed, empty if absent") String title,
    @JsonPropertyDescription("Key points supported by the document") List<String> keyPoints) {}

ExtractedContent content = chatClient.prompt()
    .user(u -> u.text(template).param("document", document))
    .advisors(StructuredOutputValidationAdvisor.builder()   // Spring AI 1.1+
        .outputType(ExtractedContent.class)
        .maxRepeatAttempts(2)                               // re-prompt with the schema violation
        .build())
    .call()
    .entity(ExtractedContent.class);                    // JSON schema derived from the record
```

> The record **is** the contract: schema for the model, type for your code.



The Atom

Task Execution

Task Execution

One Template In, One Typed Record Out



The **simplest agentic pattern** and the **building block** of every pattern that follows.

Task Execution

Expose a Task as a `Function<Request, Response>`

```
Function<DocumentRequest, ExtractedContent> extractor = request ->
    chatClient.prompt()
        .user(u -> u
            .text(new ClassPathResource("prompts/extract-document.st"))
            .param("document", request.document()))
        .call()
        .entity(ExtractedContent.class);

ExtractedContent content = extractor.apply(request);
```

A typed input/output interface makes the task **testable** and **composable**.

Task Execution

The LLM Reads, Code Decides

```
public ExtractionResult extract(DocumentRequest request) {
    ExtractedContent content = extractor.apply(request);         // LLM: read any layout
    List<String> issues = contentValidator.validate(content);    // Code: required fields, allowed values
    return new ExtractionResult(content, issues,
        issues.isEmpty() ? "ACCEPTED" : "REVIEW_REQUIRED");     // Code: the business decision
}
```

- The model **extracts** information, Java **checks** application rules
- Every pattern that follows: **code gates** between LLM steps

> Trust, but verify: validate model output before acting on it.

Task Execution

Validate the Input Before You Spend Tokens

```java
public record DocumentRequest(@NotBlank String document) { }

@PostMapping("/documents")
ExtractionResult extract(@Valid @RequestBody DocumentRequest request) {
    return service.extract(request);
}
```

```
POST /api/extraction/documents   {"document": "  "}

400 Bad Request
{ "title": "Invalid request", "detail": "document must not be blank" }
```

- The request record is the **typed contract in**: Bean Validation guards it
- `@RestControllerAdvice` : a **400** naming the fields, **zero tokens** spent

Task Execution

Boring on Purpose

- **Scope** | start big, split only when results get unreliable
- **Sampling settings** | tune for consistency and evaluate repeatability
- **Model choice** | use the simplest model that meets the quality target
- **Cap `maxTokens`** | when you know the output size
- **Tools** | extend the task, don't bloat the prompt

> A good Task Execution agent is **boring**: same input, same output shape.



The Patterns

From Controllability to Autonomy

Stage Map

From Controllability to Autonomy



The Pattern Catalogue

Every Pattern Has a Place on the Map

Mono

LLM picks the tools

- Task Execution
  - tools in the agentic loop

Graph

Your code picks the edges

- Chain Workflow
- Parallelization
- Evaluator-Optimizer
- Routing Workflow

Orchestrator

One LLM picks the agents

- Agent as Tool
- Orchestrator-Workers

Swarm

Every peer picks the next

- Peer handoffs

Built from the same atom: **Task Execution** = Prompt Template + Structured Output

Stage 1 | Mono

One Agent, One Loop

Stage 1 | Mono

One Persona, All Tools



When Is Mono Correct?

Start Here

- **One persona**, one responsibility
- **A focused toolset** for one responsibility
- **No complex orchestration** required
- **No decomposition** needed

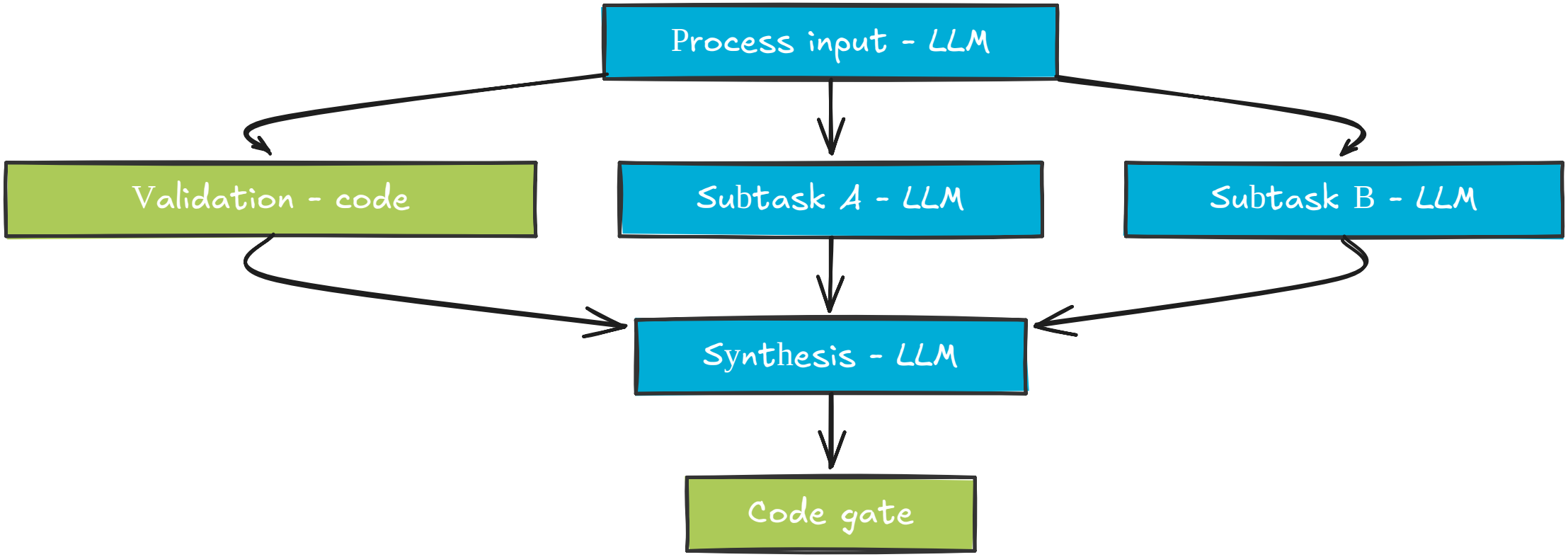
> Agentic loop = **1 loop**. How far it scales depends on your **model**.

Stage 2 | Graph

Your Code Decides the Flow

Stage 2 | Graph

The Topology Decides Who Runs



Stage 2 | Graph

Four Shapes Every Graph Is Made Of

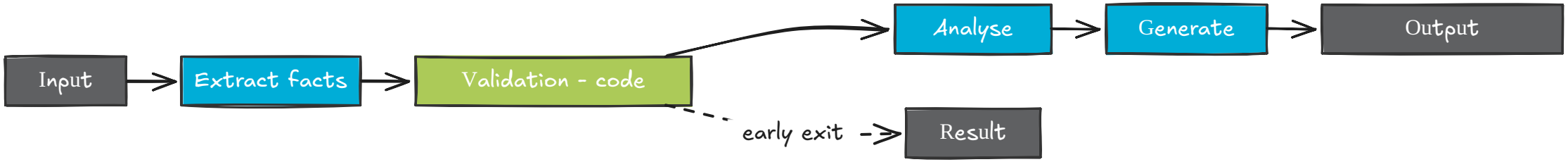
Independent steps form a **fan-out**; combining their results forms a **fan-in**. These four shapes compose into larger graphs.

Shape	Graph form	Who picks the next edge?
Chain Workflow	`A → B → C`	Your code
Parallelization	Fan-out, fan-in	Your code
Evaluator-Optimizer	Bounded cycle	Your code, on an LLM verdict
Routing Workflow	One of N branches	**The LLM**

> Watch the last column: control shifts to the model.

Graph Shape | Chain Workflow

Split a Task Into a Sequence of Steps



Use it for

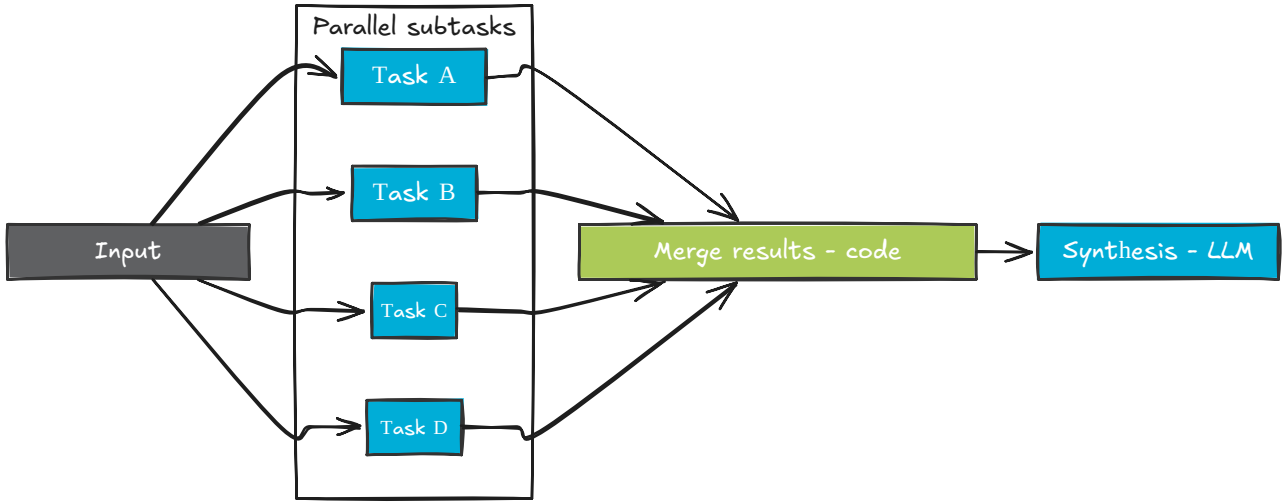
- Fixed, ordered steps
- Each step simpler than the whole
- Trade latency for accuracy

Watch out

- Latency adds up with every step
- Errors propagate: add **code gates** between steps

Graph Shape | Parallelization

Fan Out, Then Assemble



Two flavours

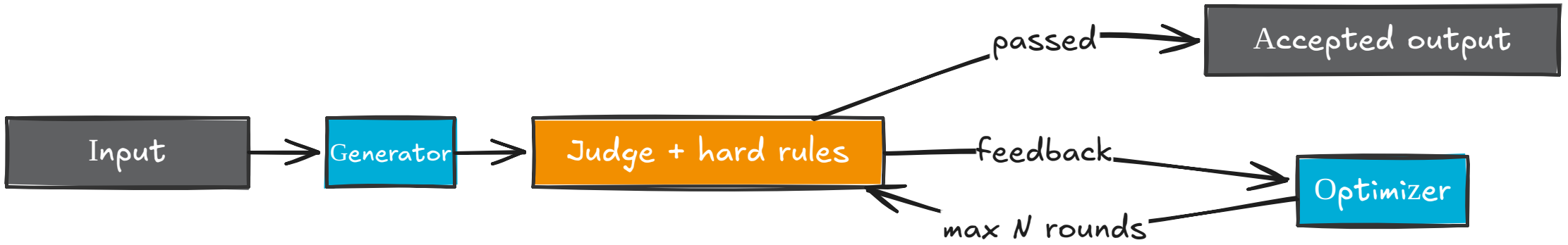
- **Sectioning** | different subtasks, results merged
- **Voting** | same task, N runs or N models, majority wins

Watch out

- Rate limits: **bound the fan-out**
- Assemble with **code** when you can, an **LLM** when you must

Graph Shape | Evaluator-Optimizer

Generate, Judge, Improve, Repeat (but Not Forever)



Use it for

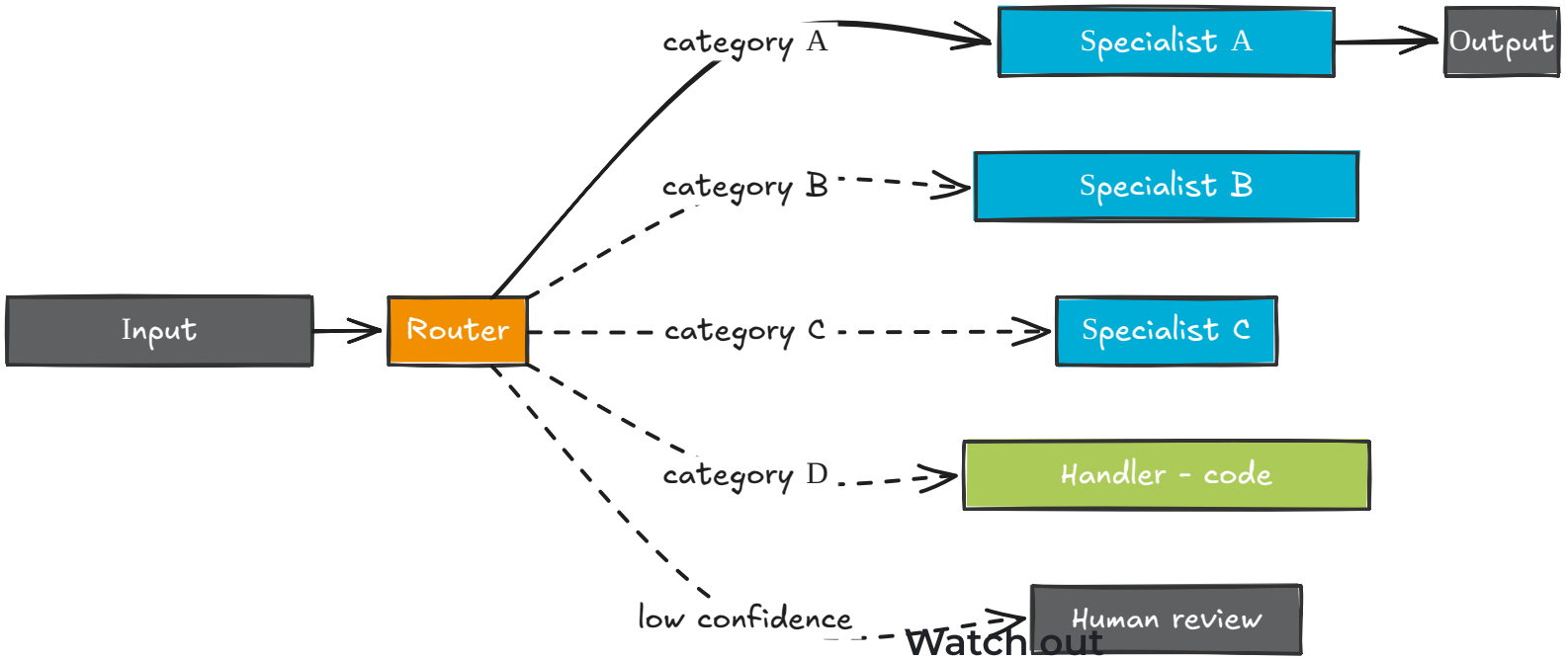
- Clear, checkable quality criteria
- Code, translations, summaries

Watch out

- **Always cap the rounds** and return the best effort
- Evaluate the **judge** against clear criteria and human assessments

Graph Shape | Routing Workflow

The LLM Picks One Branch



Use it for

- Distinct categories, each better handled on its own
- Cheap model for easy cases, strong model for hard ones

Watch out

- Describe each route's **purpose** and **expected input** clearly
- Log the **selected route** and rationale for review

When Is Graph Correct?

Deterministic Flow, Parallel Work

- **Predictable** and **auditable** flow
- **Parallel** work where steps are independent
- Built from four shapes: **chain**, **fan-out**, **bounded loop**, **routed branch**

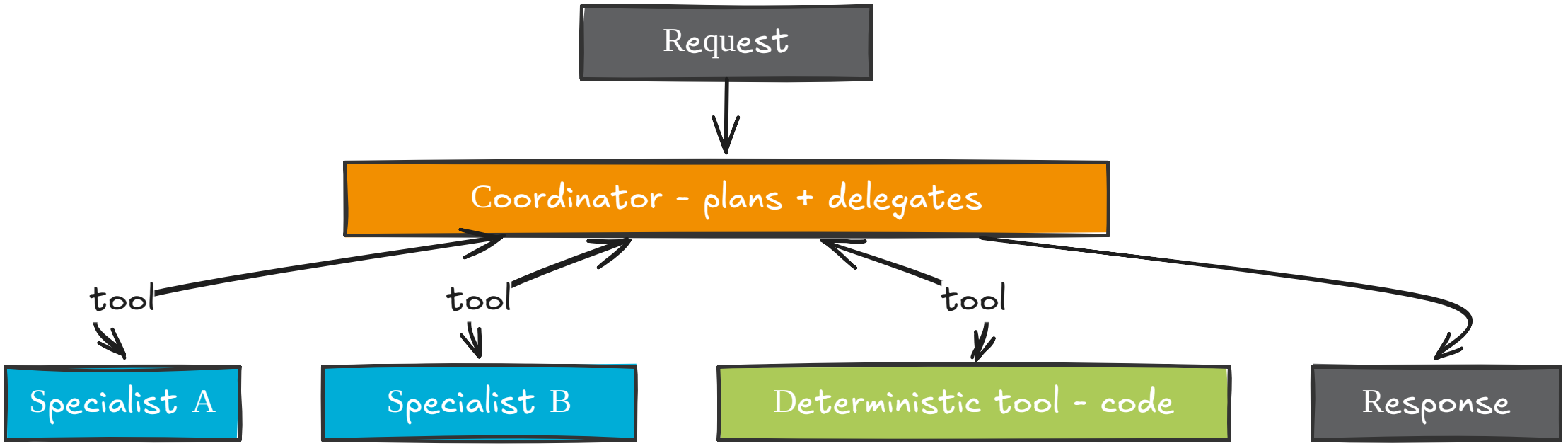
Agentic loop = **N independent loops** with merge points.

Stage 3 | Orchestrator

One Coordinator Delegates to Specialists

Stage 3 | Orchestrator

Agents as Tools

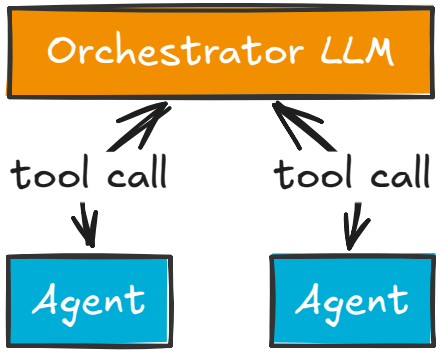


Routing let the LLM pick **one** edge. The orchestrator lets it pick **all** of them.

Stage 3 | Orchestrator

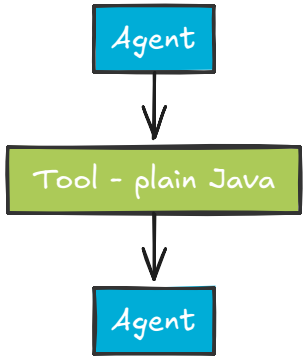
Agent as Tool ↔ Tool as Agent

Agent as Tool



The LLM **calls an agent** like any other tool whenever *it* decides to. No fixed process.

Tool as Agent



A deterministic tool **joins a workflow** as a node. Not every step needs an LLM.

Both can expose a **common input/output interface**.

When Is Orchestrator Correct?

Dynamic Flow, Central Control

- **No fixed flow**: the plan depends on the input
- **Specialists** worth keeping separate
- **Clean audit trail** through one coordinator

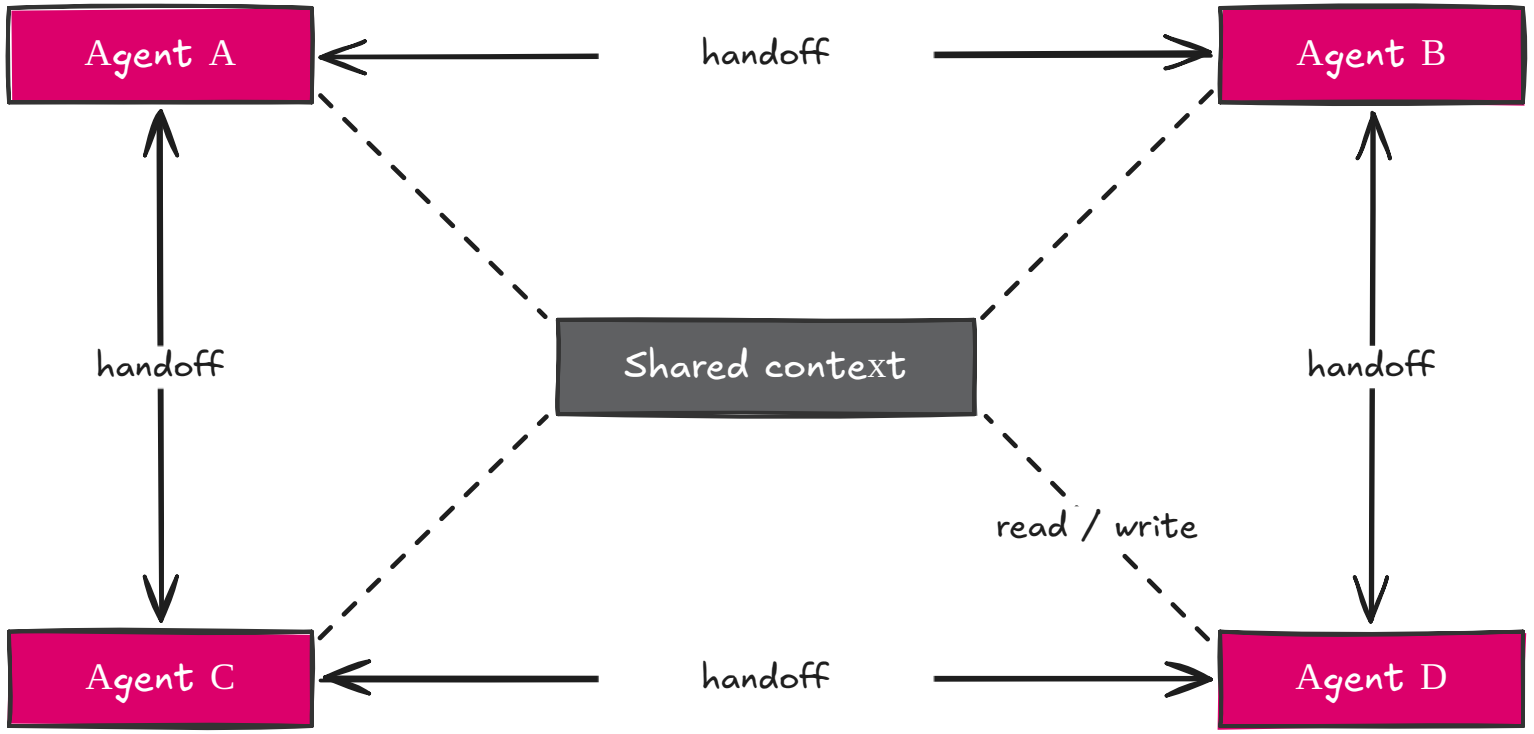
Agentic loop = **1 outer coordinator + N inner specialists.**

Stage 4 | Swarm

Peer Agents Hand Off to Each Other

Stage 4 | Swarm

Agents Decide Who Runs Next



Each **peer is an agent**; handoffs distribute coordination across the swarm.

When Is Swarm Correct?

Maximum Autonomy

- **Distributed decisions** across specialised agents
- **Domain handoffs** between peers
- **Maximum autonomy**, minimum predictability

Agentic loop = **chained loops** with peer-decided handoffs.

The Tests

Non-Deterministic Agents, Deterministic Tests



Test the Code Around the Model

Most of an Agentic System Is Plain Java

Layer	What you test	Model needed?	Test approach
Code gates and tools	Business rules, policy, overrides	No	Unit tests for validators and deterministic tools
Pattern engines	Scheduling, bounds, fallbacks, guard rails	No, scripted functions	Scripted steps with known outputs
Input contracts	Bad input is a 400, model never called	No, mocked	`@WebMvcTest`
Wiring and prompts	Templates render, schema parses, gates run	No, a canned `ChatModel`	`@SpringBootTest + MockMvc`
Provider profiles	Bedrock, OpenAI, Anthropic, Ollama boot and call tools	No, a local HTTP stub	Profile-specific integration tests
Model behaviour	Quality on real inputs	**Yes**	Representative and adversarial inputs; quality metrics and traces

> Keep automated checks **offline** with model doubles and provider stubs. Evaluate model quality separately with a real model.



The Decision

Which Pattern When?

The Four Stages at a Glance

Who Decides What Runs Next?

Stage	Who decides the flow	Agentic loop	Patterns	Pick it when
Mono	The single agent	1 loop	Task Execution + tools	One responsibility, focused tools, no decomposition
Graph	The topology (your code)	N independent loops with merge points	Chain, Parallelization, Evaluator-Optimizer, Routing	Predictable, auditable, parallel
Orchestrator	A coordinator agent	1 outer coordinator + N inner specialists	Agent as Tool, Orchestrator-Workers	No fixed flow, clean audit trail
Swarm	The peer agents	Chained loops with peer-decided handoffs	Peer handoffs	Distributed decisions and handoffs between specialists

Patterns Compose

Compose Patterns Through Common Input/Output Interfaces

- **Chain** + **Tool as Agent** gate: validation can stop the sequence early
- **Graph** of **Task Execution** agents and **code**: parallel analysis with deterministic checks
- **Orchestrator** whose workers are **Mono** agents with private tools
- **Routing** whose branches are specialist agents or **deterministic handlers**

> Start with the **simplest shape that works**. Compose, don't rewrite.

Key Takeaways

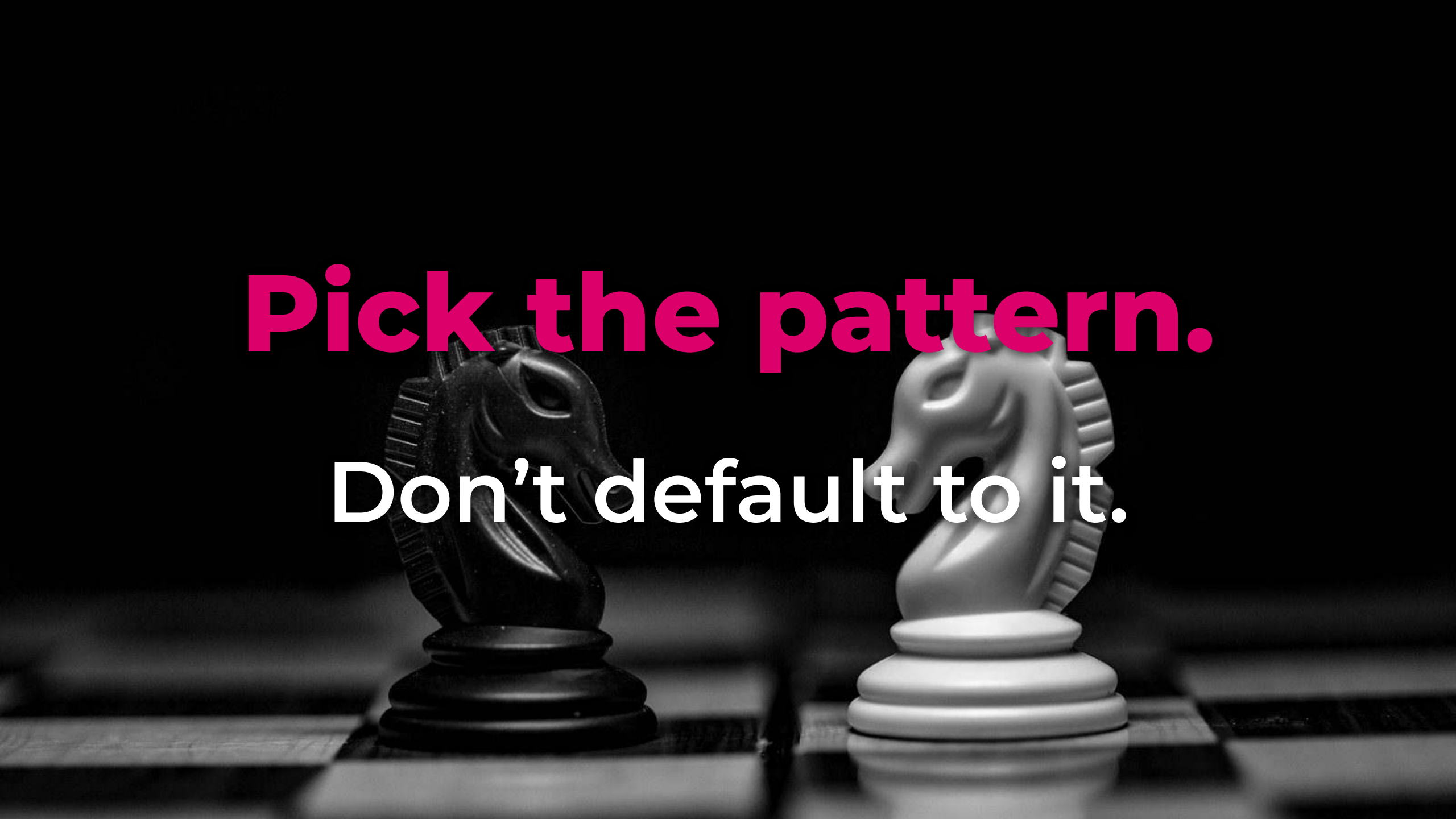
What to Remember

- **Agents** are a loop of model and tool calls
- **MCP** connects agents to tools, **A2A** connects agents to agents
- **Prompt Template + Structured Output** turn LLM calls into typed contracts
- **Task Execution** is the atom every pattern is built from
- **The LLM reads, code decides**: keep policy and hard rules in Java
- Patterns range from **controllability** (Mono, Graph) to **autonomy** (Orchestrator, Swarm)
- Every step towards autonomy costs **predictability** and needs **guard rails**
- **Test control flow** with model doubles; evaluate behaviour with real models
- **Start with Mono**; move right on the stage map only when you have to



Q&A

Let's Talk About Your Agents!


Pick the pattern.
Don’t default to it.

Designing AI-Driven Enterprise Systems

Agentic Patterns with Spring AI



Patrick Baumgartner
42talents, Zürich, Switzerland

Arnaud Jean
AWS

bitly

Further Resources

Learn and Connect

- **Companion code**: github.com/42talents/presentation-agentic-patterns-with-spring-ai
- **Spring AI Reference**: https://docs.spring.io/spring-ai/reference/
- **Spring AI Community**: https://github.com/spring-ai-community
- **Spring AI Recipes**: https://github.com/habuma/spring-ai-recipes
- **Model Context Protocol**: https://modelcontextprotocol.io
- **Agent2Agent Protocol**: https://a2a-protocol.org
- **Code Mode**: https://blog.cloudflare.com/code-mode/, https://www.anthropic.com/engineering/code-execution-with-mcp
- **Amazon Bedrock**: https://aws.amazon.com/bedrock/
- **Building Effective Agents**: https://www.anthropic.com/engineering/building-effective-agents